6.Experiment: Data Preprocessing and Machine Learning using PySpark MLlib (Pipelines)

AIM

To perform **data preprocessing** and implement **machine learning (classification/regression)** using **MLlib pipelines** in Apache Spark.

DESCRIPTION

PySpark MLlib

MLlib is Spark's scalable machine learning library that supports:

- Classification
- Regression
- Clustering
- Feature engineering

ML Pipeline Concept

Inspired by Scikit-learn pipelines, Spark ML pipelines consist of:

- **Transformer** → Transforms data (e.g., StringIndexer, VectorAssembler)
- **Estimator** → Learns from data (e.g., Logistic Regression)
- **Pipeline** → Chain of stages (transformers + estimators)

Key Components Used

- **StringIndexer** → Converts categorical data to numeric
- **VectorAssembler** → Combines features into a vector
- **Classifier/Regressor** → Model training (e.g., Logistic Regression)

STEP-BY-STEP EXPLANATION

Step 1: Import Libraries

Import PySpark ML and SQL libraries.

Step 2: Create Spark Session

Initialize Spark environment.

Step 3: Load/Create Dataset

Create a sample dataset with categorical and numerical features.

Step 4: Data Preprocessing

- Convert categorical columns using StringIndexer
- Combine features using VectorAssembler

Step 5: Define Model

- Use **Logistic Regression** for classification
  (or Linear Regression for regression tasks)

Step 6: Create Pipeline

- Chain preprocessing + model into a pipeline

Step 7: Train Model

- Fit pipeline on training data

Step 8: Make Predictions

- Apply model on test data

Step 9: Evaluate Model

- Check accuracy or performance

SOURCE CODE (PySpark MLlib Pipeline – Classification)

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

# Step 1: Create Spark Session
spark = SparkSession.builder \
    .appName("MLlib Pipeline Example") \
    .getOrCreate()

# Step 2: Create Sample Data
data = [
    ("Alice", "Female", 25, 0),
    ("Bob", "Male", 30, 1),
    ("Cathy", "Female", 22, 0),
    ("David", "Male", 35, 1),
    ("Eva", "Female", 28, 0)
]

columns = ["Name", "Gender", "Age", "Label"]
df = spark.createDataFrame(data, columns)

# Step 3: Preprocessing
indexer = StringIndexer(inputCol="Gender", outputCol="GenderIndex")
assembler = VectorAssembler(
    inputCols=["GenderIndex", "Age"],
    outputCol="features"
)

# Step 4: Define Model
lr = LogisticRegression(featuresCol="features", labelCol="Label")

# Step 5: Create Pipeline
pipeline = Pipeline(stages=[indexer, assembler, lr])

# Step 6: Train Model
model = pipeline.fit(df)

# Step 7: Make Predictions
predictions = model.transform(df)

# Step 8: Show Results
```

```
predictions.select("Name", "Gender", "Age", "features", "prediction").show()

# Stop Spark
spark.stop()
```

OUTPUT

```
+------+-------+---+-------------+----------+
| Name |Gender |Age| features    |prediction|
+------+-------+---+-------------+----------+
|Alice |Female |25 |[0.0,25.0]   |0.0       |
|Bob   |Male   |30 |[1.0,30.0]   |1.0       |
|Cathy |Female |22 |[0.0,22.0]   |0.0       |
|David |Male   |35 |[1.0,35.0]   |1.0       |
|Eva   |Female |28 |[0.0,28.0]   |0.0       |
+------+-------+---+-------------+----------+
```

RESULT

Successfully performed:

- Data preprocessing using **StringIndexer** and **VectorAssembler**
- Built a **machine learning pipeline**
- Implemented **classification using Logistic Regression**
- Generated predictions using PySpark MLlib

Thus, **MLlib pipelines for scalable machine learning** were successfully implemented and verified.